```
==================== app.py ====================

"""Qoneqt AI Video Agent - TEAM CHAOS (LJ University - LJIIET)
Qoneqt x CTRL FREAK 2026 Challenge: an LLM-powered content pipeline for the Qoneqt
Global Feed.

Run locally:   streamlit run app.py
"""
from __future__ import annotations

import io
import os
import zipfile

import streamlit as st

from agent.llm import INPUT_TYPES, LENGTHS, SYSTEM_PROMPT, TONES,
available_provider
from agent.pipeline import PIPELINE_STEPS, cleanup_old_outputs, run_pipeline
from agent.tts import VOICES

st.set_page_config(page_title="Qoneqt AI Video Agent | TEAM CHAOS", page_icon="🎬",
                   layout="wide", initial_sidebar_state="expanded")

CSS = """
<style>
.block-container {padding-top: 1.6rem;}
.hero {padding: 1.1rem 1.4rem; border-radius: 18px;
      background: linear-gradient(120deg, #0B1630 0%, #13284f 55%, #3a1d05 100%);
      border: 1px solid #22335a; margin-bottom: 1rem;}
.hero h1 {margin: 0; font-size: 2.0rem;}
.hero p {margin: .3rem 0 0 0; color: #c9d4ea;}
.badge {display:inline-block; padding: 2px 10px; border-radius: 999px; font-size: .78rem;
       font-weight: 700; margin-right: 6px;}
.b-orange {background:#FF7A00; color:#0B1630;}
.b-green {background:#16a34a; color:white;}
.b-yellow {background:#facc15; color:#1a1a1a;}
.pipe {display:flex; flex-wrap:wrap; gap:6px; align-items:center; margin:.4rem 0 .8rem 0;}
.step {padding:9px 12px; border-radius:12px; border:1px solid #2a3a5e; background:#101a30;
      min-width:104px; text-align:center; font-size:.82rem; line-height:1.25;}
.step b {display:block; font-size:.86rem;}
```

```
.step.done {border-color:#22c55e; background:#0e2a1b;}
.step.running {border-color:#FF7A00; background:#2b1a08; animation: pulse 1s infinite;}
.step.manual {border-color:#38bdf8; background:#0a2232;}
.step.error {border-color:#ef4444; background:#2d0d0d;}
.arrow {color:#FF7A00; font-weight:800;}
@keyframes pulse {0%{opacity:1} 50%{opacity:.55} 100%{opacity:1}}
.small {color:#9fb0cf; font-size:.85rem;}
</style>
"""
st.markdown(CSS, unsafe_allow_html=True)

ICONS = {"pending": "⚪", "running": "⏳", "done": "✅", "manual": "👤", "error": "❌"}


def pipeline_html(steps: list[dict] | None) -> str:
    steps = steps or [{"name": n, "status": "pending", "seconds": None, "detail": ""}
                      for n in PIPELINE_STEPS]
    parts = []
    for i, s in enumerate(steps):
        if s["status"] == "manual":
            t = "you do this"
        elif s["seconds"] is not None:
            t = f"{s['seconds']:.2f}s"
        elif s["status"] == "running":
            t = "working..."
        else:
            t = "waiting"
        parts.append(f"<div class='step {s['status']}' title='{s.get('detail', '')}'>"
                     f"<b>{ICONS[s['status']]} {s['name']}</b>{t}</div>")
        if i < len(steps) - 1:
            parts.append("<span class='arrow'>→</span>")
    return "<div class='pipe'>" + "".join(parts) + "</div>"


def show_pipeline(container, steps):
    container.markdown(pipeline_html(steps), unsafe_allow_html=True)


def upload_steps_md() -> str:
    return """
1. Click **⬇️ Download MP4** to save the video to your phone or laptop.
2. Open the **Qoneqt** app (or qoneqt.com) and log in with your team account.
3. Tap the **Create / ＋ (new post)** button.
```

4. Choose **Video** and select the MP4 you just downloaded.
5. Copy the **caption + hashtags** above (copy icon on the top-right of the box) and paste them into the post text.
6. Make sure the post is **public** so it appears on the **Global Feed**, then tap **Post**.
7. Open your post, copy its link and keep it for the hackathon submission form.
"""

```
def show_result(res: dict, key: str, compact: bool = False) -> None:
    """Video + download + Ready-for-Qoneqt section for one generated video."""
    if res.get("demo_mode"):
        st.warning(f"🟡 **{res['llm']['provider']}** - this script was written by the built-in "
                   "template generator. Add a free `GROQ_API_KEY` for real LLM scripts (see README).")
    vcol, icol = st.columns([1, 1.35], gap="large")
    with vcol:
        st.video(res["video_path"])
        with open(res["video_path"], "rb") as fh:
            st.download_button("⬇️ Download MP4", fh.read(), file_name=res["video_name"],
                               mime="video/mp4", key=f"dl_{key}", on_click="ignore",
                               type="primary", width="stretch")
        st.caption(f"{res['duration']:.1f}s · 1080x1920 (9:16) · H.264 + AAC · "
                   f"made in {res['total_seconds']:.0f}s")
    with icol:
        sc = res["script"]
        st.markdown(f"### {sc['title']}")
        st.markdown(f"<span class='badge b-orange'>LLM: {res['llm']['provider']}</span>"
                    f"<span class='badge b-green'>Voice: {res['tts_engine']}</span>"
                    f"<span class='badge b-yellow'>{res['tone']} · {res['length']}s</span>",
                    unsafe_allow_html=True)
        st.markdown("#### ✅ Ready for Qoneqt")
        st.caption("Qoneqt has no publishing API, so this agent does NOT auto-post. "
                   "Copy the text below and upload the video manually.")
        st.markdown("**Caption + hashtags (one click copy):**")
        st.code(res["caption_block"], language=None, wrap_lines=True)
        st.markdown("**Hashtags only:**")
        st.code(" ".join(sc["hashtags"]), language=None, wrap_lines=True)
        if not compact:
            with st.expander("📤 How to upload this video to Qoneqt (manual, step by step)",
                             expanded=True):
                st.markdown(upload_steps_md())
        with st.expander("📝 Full script (hook, scenes, call to action)"):
            st.markdown(f"**Hook:** {sc['hook']}")
```

```
            for i, scene in enumerate(sc["scenes"], 1):
                st.markdown(f"**Scene {i} - {scene['on_screen_text']}**  \n{scene['narration']}")
            st.markdown(f"**Call to action:** {sc['call_to_action']}")
        if res.get("notes"):
            with st.expander("ℹ️ Agent notes"):
                for n in res["notes"]:
                    st.write("- " + n)


def run_with_ui(topic, tone, length, input_type, voice, ai_images, live_box):
    def on_step(steps):
        st.session_state["live_steps"] = steps
        show_pipeline(live_box, steps)

    return run_pipeline(topic, tone, length, input_type=input_type, voice=voice,
                        ai_images=ai_images, on_step=on_step)


# ---------------------------------------------------------------- sidebar
cleanup_old_outputs()
provider, model = available_provider()
with st.sidebar:
    st.markdown("## 🎬 Qoneqt AI Video Agent")
    st.caption("TEAM CHAOS · LJ University (LJIIET)")
    if provider.startswith("Demo"):
        st.warning("🟡 **Demo mode (no API key)**\n\nScripts come from a built-in template. "
                   "Add `GROQ_API_KEY` in Streamlit *Secrets* to use the real LLM.")
    else:
        st.success(f"🟢 LLM connected: **{provider}**\n\nModel: `{model}`")
    st.markdown("### ⚙️ Video settings")
    voice_label = st.selectbox("Voiceover voice", list(VOICES.keys()), index=0)
    voice = VOICES[voice_label]
    ai_images = st.toggle("AI background images (multimodal)", value=True,
                          help="Uses the free Pollinations.ai text-to-image model with the LLM's "
                               "visual prompts. If it is slow or offline, styled gradient slides "
                               "are used instead.")
    st.markdown("---")
    st.markdown("**Pipeline**  \nTopic → LLM Script → Voice → Visuals → Video → Download
→ "
                "Manual upload to Qoneqt")
    st.caption("Qoneqt has no publishing API - videos are uploaded manually.")

st.markdown(
```

```
    "<div class='hero'><h1>🎬 Qoneqt AI Video Agent</h1>"
    "<p>An LLM-powered content pipeline that turns a <b>topic, prompt, idea or trend</b> into a
"
    "ready-to-publish short video for the <b>Qoneqt Global Feed</b> - again and again.</p>"
    "<p class='small'>Qoneqt × CTRL FREAK 2026 · TEAM CHAOS · LJ University
(LJIIET)</p></div>",
    unsafe_allow_html=True)

tab_gen, tab_batch, tab_llm, tab_about = st.tabs(
    ["🎬 Generate", "📦 Batch / Auto mode", "🧠 How the LLM works", "ℹ️ About"])

# ------------------------------------------------------------ Generate
with tab_gen:
    with st.form("gen_form"):
        c1, c2 = st.columns([2, 1], gap="large")
        with c1:
            input_type = st.radio("What are you giving the agent?", INPUT_TYPES, horizontal=True)
            topic = st.text_area("Your topic / prompt / idea / trend",
                                 value="3 tips to spot fake news", height=100,
                                 help="Examples: 'How to save money as a college student', "
                                      "'Why sleep matters before exams', 'AI tools trending in 2026'")
        with c2:
            tone = st.selectbox("Tone", TONES)
            length = st.select_slider("Video length (seconds)", options=LENGTHS, value=45)
        submitted = st.form_submit_button("🚀 Generate video", type="primary", width="stretch")

    st.markdown("#### 🔴 Live pipeline")
    live_box = st.empty()
    show_pipeline(live_box, st.session_state.get("live_steps"))

    if submitted:
        if not topic.strip():
            st.error("Please type a topic first.")
        else:
            with st.status("🤖 The agent is working...", expanded=False) as status:
                try:
                    res = run_with_ui(topic, tone, length, input_type, voice, ai_images, live_box)
                    st.session_state["last"] = res
                    status.update(label=f"✅ Video ready in {res['total_seconds']:.0f}s",
                                  state="complete")
                except Exception as exc:  # never show a raw crash to the judges
                    status.update(label="❌ Something went wrong", state="error")
                    st.error(f"The pipeline hit an error: {exc}\n\nTry again, pick a shorter "
```

```
                "length, or turn off AI background images in the sidebar.")

    if st.session_state.get("last"):
        st.markdown("---")
        show_result(st.session_state["last"], key="single")
    else:
        st.info("👆 Type a topic and press **Generate video**. The agent writes the script with an "
                "LLM, records a voiceover, designs 9:16 slides with captions and builds an MP4.")

# ---------------------------------------------------------------- Batch
with tab_batch:
    st.markdown("### 📦 Batch / Auto mode - content at scale")
    st.write("Paste several topics (one per line). The agent runs the full pipeline for each "
             "topic in turn and gives you one ready-to-upload video per topic.")
    with st.form("batch_form"):
        topics_text = st.text_area(
            "Topics (one per line, max 10)", height=150,
            value="Why sleep matters before exams\nHow to save money as a college student\n"
                  "3 signs a message is a scam")
        b1, b2 = st.columns(2)
        b_tone = b1.selectbox("Tone for all videos", TONES, key="b_tone")
        b_len = b2.select_slider("Length for all videos", options=LENGTHS, value=30,
key="b_len")
        b_go = st.form_submit_button("🚀 Generate all videos", type="primary", width="stretch")

    if b_go:
        topics = [t.strip() for t in topics_text.splitlines() if t.strip()][:10]
        if not topics:
            st.error("Add at least one topic.")
        else:
            results = []
            bar = st.progress(0.0, text="Starting...")
            b_live = st.empty()
            for i, t in enumerate(topics):
                bar.progress(i / len(topics), text=f"Video {i + 1} of {len(topics)}: {t}")
                try:
                    results.append(run_with_ui(t, b_tone, b_len, "Topic", voice, ai_images, b_live))
                except Exception as exc:
                    st.error(f"'{t}' failed: {exc}")
            bar.progress(1.0, text=f"Done: {len(results)} of {len(topics)} videos ready")
            st.session_state["batch"] = results
            if results:
                st.session_state["last"] = results[-1]
```

```
    batch = st.session_state.get("batch") or []
    if batch:
        buf = io.BytesIO()
        with zipfile.ZipFile(buf, "w", zipfile.ZIP_STORED) as zf:
            for r in batch:
                zf.write(r["video_path"], r["video_name"])
                zf.writestr(r["video_name"].replace(".mp4", "_caption.txt"), r["caption_block"])
        st.download_button(f"⬇️ Download all {len(batch)} videos + captions (ZIP)", buf.getvalue(),
                           file_name="qoneqt_batch_videos.zip", mime="application/zip",
                           on_click="ignore", key="dl_zip")
        for i, r in enumerate(batch):
            with st.expander(f"🎞️ {i + 1}. {r['script']['title']}  ({r['duration']:.0f}s)",
                             expanded=(i == 0)):
                show_result(r, key=f"batch_{i}", compact=True)

    st.markdown("---")
    st.markdown("#### ⏰ How this runs automatically every day")
    st.write("The same pipeline has a headless runner (`run_pipeline.py`). A scheduler such as a "
             "**GitHub Actions cron job** can call it every morning with a list of topics, then "
             "save the videos as downloadable artifacts. A team member downloads them and uploads "
             "to Qoneqt (Qoneqt has no publishing API, so posting stays manual). A ready-made "
             "example is in `docs/daily-video.yml.example`:")
    st.code("""on:
  schedule:
    - cron: "30 3 * * *"   # every day at 9:00 AM IST
jobs:
  make-videos:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: sudo apt-get install -y ffmpeg && pip install -r requirements.txt
      - run: python run_pipeline.py --topics-file topics.txt
        env: { GROQ_API_KEY: "${{ secrets.GROQ_API_KEY }}" }
      - uses: actions/upload-artifact@v4
        with: { name: daily-videos, path: outputs/**/*.mp4 }""", language="yaml")

# ---------------------------------------------------------------- How the LLM works
with tab_llm:
```

```
    st.markdown("### 🧠 How the LLM works - live view of the last run")
    last = st.session_state.get("last")
    if not last:
        st.info("Generate a video first - this panel then shows the exact prompts, the raw LLM "
                "response and the time each pipeline step took.")
        show_pipeline(st.container(), None)
    else:
        st.markdown(f"**Topic:** {last['topic']}  ·  **LLM:** {last['llm']['provider']} "
                    f"(`{last['llm']['model']}`)  ·  **Total time:** {last['total_seconds']:.1f}s")
        if last.get("demo_mode"):
            st.warning(f"🟡 {last['llm']['provider']}: the response below came from the built-in "
                       "template generator, which returns the same JSON format as the LLM.")
        show_pipeline(st.container(), last["steps"])
        rows = [{"Step": s["name"], "Status": f"{ICONS[s['status']]} {s['status']}",
                 "Time (s)": ("you do this" if s["status"] == "manual"
                              else f"{s['seconds']:.2f}" if s["seconds"] is not None else "-"),
                 "Details": s["detail"]} for s in last["steps"]]
        st.dataframe(rows, hide_index=True, width="stretch")
        p1, p2 = st.columns(2, gap="large")
        with p1:
            st.markdown("**1️⃣ System prompt (the agent's rules)**")
            st.code(last["llm"]["system_prompt"], language=None, wrap_lines=True)
        with p2:
            st.markdown("**2️⃣ User prompt (built from your inputs)**")
            st.code(last["llm"]["user_prompt"], language=None, wrap_lines=True)
            st.markdown("**3️⃣ Raw LLM response**")
            st.code(last["llm"]["raw_response"], language="json", wrap_lines=True)
        st.markdown("**4️⃣ Parsed & validated script (what the video is built from)**")
        st.json(last["script"], expanded=False)
        if last.get("slides"):
            st.markdown("**5️⃣ Generated slides**")
            cols = st.columns(min(len(last["slides"]), 7))
            for c, s in zip(cols, last["slides"]):
                c.image(s)

    st.markdown("---")
    st.markdown("""
### 📚 In plain English

**What is an LLM?**
A Large Language Model (LLM) is an AI trained on huge amounts of text. It predicts the next words,
which lets it write, summarise and plan like a human writer. Our agent uses **Llama 3.3 70B**
```

(an open model by Meta) served for free and very fast by **Groq**. Google **Gemini** can be used as
an alternative.

What does the LLM do in our agent?
It is the *brain* of the pipeline. From one short topic it writes the complete plan for a video:
a scroll-stopping **hook**, 3-5 **scenes** (what the voice says + the headline on screen + a
picture idea for the image model), a **call to action**, and a Qoneqt **caption with hashtags**.

How does prompting work?
We send two messages. The **system prompt** gives the LLM its job and rules (Qoneqt Global Feed,
9:16 short video, simple English, stay on topic, word budget). The **user prompt** is built from
your inputs: topic, tone, length and number of scenes. Changing the tone or length changes the
prompt - and the script.

Why structured JSON output?
We ask the LLM to answer *only* in JSON with fixed fields (`title`, `hook`, `scenes`,
`call_to_action`, `caption`, `hashtags`). JSON is easy for code to read, so the rest of the
pipeline can automatically turn each scene into a voiceover, a slide and a video clip - with no
human copy-pasting. The app then **validates** the JSON (enough scenes, short headlines, clean
hashtags) and retries or falls back if anything is wrong, so the pipeline never breaks.

After the LLM
Voice: neural text-to-speech (edge-tts, gTTS backup) → Visuals: AI images from the LLM's visual
prompts (Pollinations) + Pillow slides with burned-in captions → Video: ffmpeg builds a 1080x1920
H.264/AAC MP4 with each slide timed to its voiceover → You download it and upload it to Qoneqt.
""")

-- About
with tab_about:
    st.markdown("""
ℹ️ About the project

Challenge: Qoneqt × CTRL FREAK 2026 - *Build an LLM-Powered Content Pipeline for Qoneqt*.
"How can AI help create high-quality, engaging video content for the Qoneqt Global Feed at scale?"

❓ Problem
Creating good short videos takes time: researching, writing a script, recording a voiceover, designing visuals, editing and adding captions. Communities on Qoneqt need a steady flow of fresh,
engaging content, and doing all of this by hand does not scale.

💡 Our solution
An **AI agent** that turns a **topic, prompt, idea or trend** into a **ready-to-publish 9:16 video** in about a minute - and can repeat it for many topics (Batch / Auto mode):

1. **Input** - topic + tone + length
2. **LLM** - writes a structured JSON script: hook, scenes, call to action, caption, hashtags
3. **Voice** - neural text-to-speech voiceover
4. **Visuals** - AI-generated background images (multimodal model) + designed slides with captions
5. **Video** - ffmpeg assembles a 1080x1920 MP4 timed to the voice
6. **Download + manual upload** - Qoneqt has no publishing API, so we provide the MP4 plus a
   copy-ready caption and step-by-step upload instructions

🛠️ Tech stack & resources
Part	Technology
Web app & deployment	Python, Streamlit, Streamlit Community Cloud, GitHub
LLM (script)	Llama 3.3 70B Versatile via Groq API (free tier); optional Google Gemini
Fallback	Built-in template generator ("Demo mode")
Voiceover	edge-tts (Microsoft neural voices), gTTS backup
Visual model	Pollinations.ai text-to-image (free, no key)
Slides & captions	Pillow (PIL), NumPy, DejaVu Sans font
Video	FFmpeg (H.264 + AAC, 30 fps, 9:16)
Automation	Headless runner + GitHub Actions cron (example)

🎯 How it meets the problem statement
- ✅ Input: topic / prompt / idea / trend
- ✅ LLM for scripting and storytelling (hook, script, scene plan)
- ✅ Visual / multimodal model + automated video composition
- ✅ Ready-to-publish video formatted for the Qoneqt feed (vertical 9:16, captions, voiceover)
- ✅ Repeatable pipeline at scale (Batch / Auto mode + scheduler)
- ✅ Deployed live; generated video published manually on the Qoneqt Global Feed

🌍 Impact
- Creators and communities can go from idea to video in about a minute instead of hours.
- Burned-in captions make videos accessible and watchable on mute.

- Consistent, branded, on-topic content keeps the Global Feed fresh and engaging.
- Free tools only - any student or small community can run it.

👥 TEAM CHAOS - LJ University (LJIIET)
Member	Role
Jenish	Captain
Jiya	Presenter
Ansh	Research & testing
Jenil	Builder
""")

========== run_pipeline.py ==========

"""Headless runner for the Qoneqt AI Video Agent (no web UI).

Examples:
  python run_pipeline.py --topic "3 tips to spot fake news"
  python run_pipeline.py --topics-file topics.txt --tone Motivational --length 30
Used for testing and for scheduled runs (e.g. a daily GitHub Actions cron job).
"""
from __future__ import annotations

import argparse
import json
import sys

from agent.llm import LENGTHS, TONES
from agent.pipeline import run_pipeline
from agent.tts import DEFAULT_VOICE

def main() -> int:
    ap = argparse.ArgumentParser(description="Turn topics into Qoneqt-ready short videos.")
    ap.add_argument("--topic", help="One topic / idea / prompt / trend")
    ap.add_argument("--topics-file", help="Text file with one topic per line (batch mode)")
    ap.add_argument("--tone", default="Informative", choices=TONES)
    ap.add_argument("--length", type=int, default=45, choices=LENGTHS)
    ap.add_argument("--voice", default=DEFAULT_VOICE)
    ap.add_argument("--ai-images", action="store_true", help="Use free AI background images")
    ap.add_argument("--out", help="Output folder (single topic only)")
    args = ap.parse_args()

```
    topics = []
    if args.topics_file:
        with open(args.topics_file, encoding="utf-8") as fh:
            topics = [line.strip() for line in fh if line.strip()]
    if args.topic:
        topics.insert(0, args.topic)
    if not topics:
        ap.error("give --topic or --topics-file")

    summary = []
    for i, topic in enumerate(topics, 1):
        print(f"[{i}/{len(topics)}] {topic}", file=sys.stderr)
        res = run_pipeline(topic, args.tone, args.length, voice=args.voice,
                           ai_images=args.ai_images,
                           out_dir=args.out if len(topics) == 1 else None,
                           on_step=lambda steps: None)
        summary.append({
            "topic": topic,
            "video": res["video_path"],
            "duration": res["duration"],
            "llm": res["llm"]["provider"],
            "tts": res["tts_engine"],
            "visuals": res["visuals_source"],
            "caption": res["caption_block"],
            "steps": {s["name"]: [s["status"], s["seconds"]] for s in res["steps"]},
            "notes": res["notes"],
            "total_seconds": res["total_seconds"],
        })
    print(json.dumps(summary, indent=2, ensure_ascii=False))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())


==================== agent/__init__.py ====================

"""Qoneqt AI Video Agent - TEAM CHAOS (LJ University - LJIIET).

An LLM-powered content pipeline: Topic -> LLM Script -> Voice -> Visuals -> Video.
"""
```

```
__version__ = "1.0.0"


==================== agent/llm.py ====================

"""The LLM brain of the agent: turns a topic into a structured JSON video script.

Provider order:
  1. Groq (free API, OpenAI-compatible) -> model llama-3.3-70b-versatile
  2. Google Gemini (optional alternative)
  3. Built-in template generator ("Demo mode (no API key)") so the demo never crashes
"""
from __future__ import annotations

import json
import re
import time
from dataclasses import dataclass, field

import requests

from .templates import template_script
from .utils import clean_text, get_secret

GROQ_URL = "https://api.groq.com/openai/v1/chat/completions"
GEMINI_URL =
"https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent"
DEFAULT_GROQ_MODEL = "llama-3.3-70b-versatile"
DEFAULT_GEMINI_MODEL = "gemini-2.5-flash"

TONES = ["Informative", "Funny", "Motivational", "Awareness"]
LENGTHS = [30, 45, 60]
INPUT_TYPES = ["Topic", "Prompt", "Idea", "Trend"]

# About 2.5 spoken words per second for a natural voiceover.
WORDS_PER_SECOND = 2.5
SCENES_FOR_LENGTH = {30: 3, 45: 4, 60: 5}

SYSTEM_PROMPT = """You are the scriptwriter inside an AI content agent that creates short, \
engaging vertical (9:16) videos for the Qoneqt Global Feed. Qoneqt is a community-first social \
platform where people discover communities, connect around shared interests, and share
content.
```

```
Your job: turn the user's topic, prompt, idea or trend into a punchy, accurate, \
family-friendly short-video script in simple English.

Rules:
- Start with a scroll-stopping HOOK (one short spoken sentence).
- Then write the requested number of SCENES. Each scene has:
    * "narration": what the voiceover says (1-2 short sentences, conversational).
    * "on_screen_text": a bold headline for the slide (max 7 words, no emojis).
    * "visual_prompt": a short description of a background image for an AI image model \
(no text in the image).
- End with a CALL TO ACTION that invites people to follow, comment or join the community on
Qoneqt.
- Write a Qoneqt post "caption" (1-2 sentences, may use 1-2 emojis) and 5-8 relevant
"hashtags".
- Stay strictly on the user's topic. Be factually careful; no medical, legal or financial guarantees.
- Respect the total word budget so the voiceover fits the video length.

Respond with ONLY valid JSON (no markdown, no commentary) using exactly this schema:
{
  "title": "string (max 8 words)",
  "hook": "string (spoken hook sentence)",
  "hook_on_screen": "string (max 7 words, no emojis)",
  "scenes": [
    {"narration": "string", "on_screen_text": "string", "visual_prompt": "string"}
  ],
  "call_to_action": "string (spoken)",
  "caption": "string",
  "hashtags": ["#Example", "#Qoneqt"]
}"""


def build_user_prompt(topic: str, tone: str, length: int, input_type: str = "Topic") -> str:
    n_scenes = SCENES_FOR_LENGTH.get(int(length), 4)
    words = int(int(length) * WORDS_PER_SECOND)
    return (
        f"{input_type}: {topic.strip()}\n"
        f"Tone: {tone}\n"
        f"Target video length: {length} seconds\n"
        f"Number of scenes: exactly {n_scenes}\n"
        f"Total spoken words (hook + all narration + call to action): about {words} words\n"
        f"Audience: young Indian and global users scrolling the Qoneqt Global Feed.\n"
        f"Return the JSON object now."
```

```
    )


@dataclass
class ScriptResult:
    script: dict
    provider: str           # "Groq", "Gemini" or "Demo mode (no API key)"
    model: str
    system_prompt: str
    user_prompt: str
    raw_response: str
    demo_mode: bool
    seconds: float = 0.0
    notes: list[str] = field(default_factory=list)


def available_provider() -> tuple[str, str]:
    """Which LLM will be used, for display in the UI."""
    if get_secret("GROQ_API_KEY"):
        return "Groq", get_secret("GROQ_MODEL", DEFAULT_GROQ_MODEL)
    if get_secret("GEMINI_API_KEY"):
        return "Gemini", get_secret("GEMINI_MODEL", DEFAULT_GEMINI_MODEL)
    return "Demo mode (no API key)", "built-in template generator"


def _call_groq(system: str, user: str, key: str, model: str) -> str:
    resp = requests.post(
        GROQ_URL,
        headers={"Authorization": f"Bearer {key}", "Content-Type": "application/json"},
        json={
            "model": model,
            "messages": [{"role": "system", "content": system}, {"role": "user", "content": user}],
            "temperature": 0.8,
            "max_tokens": 1800,
            "response_format": {"type": "json_object"},
        },
        timeout=60,
    )
    if resp.status_code != 200:
        raise RuntimeError(f"Groq HTTP {resp.status_code}: {resp.text[:300]}")
    return resp.json()["choices"][0]["message"]["content"]
```

```
def _call_gemini(system: str, user: str, key: str, model: str) -> str:
    resp = requests.post(
        GEMINI_URL.format(model=model),
        headers={"x-goog-api-key": key, "Content-Type": "application/json"},
        json={
            "systemInstruction": {"parts": [{"text": system}]},
            "contents": [{"role": "user", "parts": [{"text": user}]}],
            "generationConfig": {"responseMimeType": "application/json", "temperature": 0.8},
        },
        timeout=60,
    )
    if resp.status_code != 200:
        raise RuntimeError(f"Gemini HTTP {resp.status_code}: {resp.text[:300]}")
    data = resp.json()
    return data["candidates"][0]["content"]["parts"][0]["text"]


def parse_json(raw: str) -> dict:
    """Parse the model output, tolerating ```json fences or extra text around the object."""
    text = raw.strip()
    text = re.sub(r"^```(?:json)?\s*|\s*```$", "", text)
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        start, end = text.find("{"), text.rfind("}")
        if start == -1 or end <= start:
            raise
        return json.loads(text[start:end + 1])


def normalize_script(data: dict, topic: str, length: int) -> dict:
    """Validate and clean the JSON so the video step always gets what it expects."""
    if not isinstance(data, dict):
        raise ValueError("LLM output is not a JSON object")
    scenes_in = data.get("scenes") or []
    scenes = []
    for sc in scenes_in:
        if not isinstance(sc, dict):
            continue
        narration = clean_text(sc.get("narration", ""))
        if not narration:
            continue
        on_screen = clean_text(sc.get("on_screen_text") or sc.get("on_screen") or narration)
```

```
        words = on_screen.split()
        if len(words) > 9:
            on_screen = " ".join(words[:9])
        scenes.append({
            "narration": narration,
            "on_screen_text": on_screen,
            "visual_prompt": clean_text(sc.get("visual_prompt") or on_screen),
        })
    if len(scenes) < 3:
        raise ValueError("LLM returned fewer than 3 usable scenes")
    scenes = scenes[:5]

    title = clean_text(data.get("title") or topic)[:80]
    hook = clean_text(data.get("hook") or f"Here is something you should know about {topic}.")
    hook_on_screen = clean_text(data.get("hook_on_screen") or title)
    cta = clean_text(data.get("call_to_action") or "Follow us on Qoneqt for more!")
    caption = str(data.get("caption") or title).strip()

    tags_in = data.get("hashtags") or []
    if isinstance(tags_in, str):
        tags_in = re.split(r"[\s,]+", tags_in)
    tags = []
    for t in tags_in:
        t = re.sub(r"[^A-Za-z0-9_]", "", str(t))
        if t and f"#{t}".lower() not in [x.lower() for x in tags]:
            tags.append(f"#{t}")
    for must in ("#Qoneqt", "#GlobalFeed"):
        if must.lower() not in [x.lower() for x in tags]:
            tags.append(must)

    return {
        "title": title,
        "hook": hook,
        "hook_on_screen": hook_on_screen,
        "scenes": scenes,
        "call_to_action": cta,
        "caption": caption,
        "hashtags": tags[:10],
        "target_seconds": int(length),
    }


def generate_script(topic: str, tone: str = "Informative", length: int = 45,
```

```
                input_type: str = "Topic") -> ScriptResult:
    """Ask the LLM for a script. Never raises: falls back to the template generator."""
    topic = clean_text(topic) or "3 tips to spot fake news"
    system = SYSTEM_PROMPT
    user = build_user_prompt(topic, tone, length, input_type)
    notes: list[str] = []
    t0 = time.time()

    attempts = []
    if get_secret("GROQ_API_KEY"):
        attempts.append(("Groq", get_secret("GROQ_MODEL", DEFAULT_GROQ_MODEL),
                         lambda m: _call_groq(system, user, get_secret("GROQ_API_KEY"), m)))
    if get_secret("GEMINI_API_KEY"):
        attempts.append(("Gemini", get_secret("GEMINI_MODEL", DEFAULT_GEMINI_MODEL),
                         lambda m: _call_gemini(system, user, get_secret("GEMINI_API_KEY"), m)))

    for provider, model, call in attempts:
        for attempt in range(2):  # one retry for flaky networks / bad JSON
            try:
                raw = call(model)
                script = normalize_script(parse_json(raw), topic, length)
                return ScriptResult(script, provider, model, system, user, raw, False,
                                    time.time() - t0, notes)
            except Exception as exc:  # keep going, never crash the demo
                notes.append(f"{provider} attempt {attempt + 1} failed: {exc}")

    if not attempts:
        notes.append("No GROQ_API_KEY or GEMINI_API_KEY found, so the built-in template "
                     "generator wrote the script.")
    script = template_script(topic, tone, length)
    raw = json.dumps(script, indent=2, ensure_ascii=False)
    script = normalize_script(script, topic, length)
    label = "Demo mode (no API key)" if not attempts else "Demo mode (LLM call failed)"
    return ScriptResult(script, label, "built-in template generator",
                        system, user, raw, True, time.time() - t0, notes)


==================== agent/pipeline.py ====================

"""The agent's orchestrator: Topic -> LLM Script -> Voice -> Visuals -> Video -> Download.

Run headless:  python run_pipeline.py --topic "3 tips to spot fake news"
"""
```

```
from __future__ import annotations

import json
import os
import shutil
import time
import uuid
from concurrent.futures import ThreadPoolExecutor
from dataclasses import asdict
from typing import Callable

from .llm import generate_script
from .tts import DEFAULT_VOICE, synthesize
from .utils import OUTPUT_DIR, media_duration, slugify
from .video import build_segment, concat_segments
from .visuals import (fetch_ai_image, make_background, render_base, render_caption_frame,
                      split_captions)

PIPELINE_STEPS = ["Topic", "LLM Script", "Voice", "Visuals", "Video", "Download",
                  "Manual upload to Qoneqt"]
GAP_SECONDS = 0.35  # small pause between scenes


class StepTracker:
    """Records status + time for each pipeline step and notifies the UI live."""

    def __init__(self, callback: Callable[[list[dict]], None] | None = None):
        self.steps = [{"name": n, "status": "pending", "seconds": None, "detail": ""}
                      for n in PIPELINE_STEPS]
        self.callback = callback
        self._t0: dict[str, float] = {}

    def _get(self, name):
        return next(s for s in self.steps if s["name"] == name)

    def _notify(self):
        if self.callback:
            try:
                self.callback([dict(s) for s in self.steps])
            except Exception:
                pass

    def start(self, name, detail=""):
```

```
        s = self._get(name)
        s.update(status="running", detail=detail)
        self._t0[name] = time.time()
        self._notify()

    def done(self, name, detail="", status="done"):
        s = self._get(name)
        s.update(status=status, detail=detail or s["detail"],
                 seconds=round(time.time() - self._t0.get(name, time.time()), 2))
        self._notify()

    def fail(self, name, detail):
        self.done(name, detail, status="error")


def build_segments(script: dict) -> list[dict]:
    """Hook slide + one slide per scene + call-to-action slide."""
    n = len(script["scenes"])
    segs = [{"label": "HOOK", "headline": script.get("hook_on_screen") or script["title"],
             "narration": script["hook"], "visual_prompt": script["title"]}]
    for i, sc in enumerate(script["scenes"], 1):
        segs.append({"label": f"SCENE {i} / {n}", "headline": sc["on_screen_text"],
                     "narration": sc["narration"], "visual_prompt": sc["visual_prompt"]})
    segs.append({"label": "FOLLOW ON QONEQT", "headline": "Follow for more on Qoneqt",
                 "narration": script["call_to_action"],
                 "visual_prompt": f"{script['title']}, community of happy people, celebration"})
    return segs


def cleanup_old_outputs(max_age_hours: float = 6.0) -> None:
    """Keep the server disk tidy (Streamlit Cloud has limited space)."""
    if not os.path.isdir(OUTPUT_DIR):
        return
    cutoff = time.time() - max_age_hours * 3600
    for name in os.listdir(OUTPUT_DIR):
        path = os.path.join(OUTPUT_DIR, name)
        try:
            if os.path.isdir(path) and os.path.getmtime(path) < cutoff:
                shutil.rmtree(path, ignore_errors=True)
        except OSError:
            pass
```

```
def run_pipeline(topic: str, tone: str = "Informative", length: int = 45,
                 input_type: str = "Topic", voice: str = DEFAULT_VOICE,
                 ai_images: bool = False, out_dir: str | None = None,
                 on_step: Callable[[list[dict]], None] | None = None) -> dict:
    """Run the whole agent once. Returns a result dict (video path, script, prompts, timings)."""
    tracker = StepTracker(on_step)
    notes: list[str] = []
    run_id = time.strftime("%Y%m%d-%H%M%S") + "-" + uuid.uuid4().hex[:6]
    work = out_dir or os.path.join(OUTPUT_DIR, run_id)
    os.makedirs(work, exist_ok=True)
    t_start = time.time()

    # 1. Topic
    tracker.start("Topic")
    topic = (topic or "").strip() or "3 tips to spot fake news"
    tracker.done("Topic", f'{input_type}: "{topic}" | tone {tone} | {length}s')

    # 2. LLM Script
    tracker.start("LLM Script", "Asking the LLM for a structured JSON script...")
    llm = generate_script(topic, tone, int(length), input_type)
    script = llm.script
    notes += llm.notes
    with open(os.path.join(work, "script.json"), "w", encoding="utf-8") as fh:
        json.dump(script, fh, indent=2, ensure_ascii=False)
    tracker.done("LLM Script", f"{llm.provider} - {llm.model} - {len(script['scenes'])} scenes")

    segments = build_segments(script)

    # 3. Voice (scenes in parallel to save time)
    tracker.start("Voice", f"Text-to-speech for {len(segments)} parts...")

    def _voice(i_seg):
        i, seg = i_seg
        path = os.path.join(work, f"voice_{i:02d}.mp3")
        engine, dur, tts_notes = synthesize(seg["narration"], path, voice)
        return i, path, engine, dur, tts_notes

    with ThreadPoolExecutor(max_workers=4) as pool:
        voice_results = sorted(pool.map(_voice, enumerate(segments)))
    engines = set()
    for i, path, engine, dur, tts_notes in voice_results:
        segments[i].update(audio=path, audio_seconds=dur)
        engines.add(engine)
```

```
            notes += tts_notes
        tts_engine = ", ".join(sorted(engines))
        tracker.done("Voice", f"{tts_engine} - voice {voice}")

        # 4. Visuals
        tracker.start("Visuals", "Rendering 1080x1920 slides with captions...")
        ai_imgs = [None] * len(segments)
        visuals_source = "Pillow gradient slides"
        if ai_images:
            with ThreadPoolExecutor(max_workers=4) as pool:
                ai_imgs = list(pool.map(lambda a: fetch_ai_image(a[1]["visual_prompt"], seed=1000 +
a[0]),
                                        enumerate(segments)))
            got = sum(1 for x in ai_imgs if x is not None)
            visuals_source = f"AI images (Pollinations.ai) for {got}/{len(segments)} slides + Pillow"
            if got < len(segments):
                notes.append(f"AI image model returned {got}/{len(segments)} images; "
                             "used gradient slides for the rest.")
        total = len(segments)
        for i, seg in enumerate(segments):
            bg = make_background(tone, i, ai_imgs[i])
            base = render_base(bg, seg["label"], seg["headline"], i, total, tone)
            base.convert("RGB").save(os.path.join(work, f"slide_{i:02d}.png"), compress_level=1)
            chunks = split_captions(seg["narration"])
            weights = [len(c) + 3 for c in chunks]
            audio = seg["audio_seconds"]
            frames = []
            for j, (chunk, wgt) in enumerate(zip(chunks, weights)):
                dur = audio * wgt / sum(weights)
                if j == len(chunks) - 1:
                    dur += GAP_SECONDS
                frame_path = os.path.join(work, f"frame_{i:02d}_{j:02d}.png")
                render_caption_frame(base, chunk).save(frame_path, compress_level=1)
                frames.append((frame_path, max(dur, 0.2)))
            seg["frames"] = frames
            seg["seconds"] = audio + GAP_SECONDS
        tracker.done("Visuals", visuals_source)

        # 5. Video
        tracker.start("Video", "Encoding H.264 + AAC with ffmpeg...")
        seg_paths = []
        for i, seg in enumerate(segments):
            p = os.path.join(work, f"segment_{i:02d}.mp4")
```

```
    build_segment(seg["frames"], seg["audio"], seg["seconds"], p)
    seg_paths.append(p)
video_name = f"qoneqt_{slugify(script['title'])}.mp4"
video_path = os.path.join(work, video_name)
concat_segments(seg_paths, video_path)
duration = media_duration(video_path)
tracker.done("Video", f"{duration:.1f}s, 1080x1920, {os.path.getsize(video_path)/1e6:.1f} MB")

# Remove intermediate files, keep slides + script + final video
for name in os.listdir(work):
    if name.startswith(("frame_", "segment_", "voice_")) or name.endswith(".txt"):
        try:
            os.remove(os.path.join(work, name))
        except OSError:
            pass

# 6. Download / 7. Manual upload
tracker.start("Download")
tracker.done("Download", "MP4 ready to download")
tracker.start("Manual upload to Qoneqt")
tracker.done("Manual upload to Qoneqt",
             "Qoneqt has no publishing API - upload the MP4 by hand", status="manual")

caption_block = script["caption"].strip() + "\n\n" + " ".join(script["hashtags"])
llm_info = asdict(llm)
llm_info.pop("script", None)
return {
    "run_id": run_id,
    "topic": topic,
    "input_type": input_type,
    "tone": tone,
    "length": int(length),
    "voice": voice,
    "script": script,
    "llm": llm_info,
    "demo_mode": llm.demo_mode,
    "tts_engine": tts_engine,
    "visuals_source": visuals_source,
    "video_path": video_path,
    "video_name": video_name,
    "duration": round(duration, 2),
    "slides": sorted(os.path.join(work, f) for f in os.listdir(work) if f.startswith("slide_")),
    "caption_block": caption_block,
```

```
        "steps": tracker.steps,
        "notes": notes,
        "total_seconds": round(time.time() - t_start, 2),
        "work_dir": work,
    }


==================== agent/templates.py ====================

"""Built-in template script generator used in "Demo mode (no API key)".

It produces the same JSON structure as the LLM so the rest of the pipeline is identical.
It is deliberately simple: the real creativity comes from the LLM when a key is set.
"""
from __future__ import annotations

import hashlib
import random
import re

from .utils import clean_text

SCENES_FOR_LENGTH = {30: 3, 45: 4, 60: 5}

HOOKS = {
    "Informative": [
        "Here is what most people get wrong about {t}.",
        "Let's break down {t} in under a minute.",
        "Quick explainer: {t}. Stay till the end.",
    ],
    "Funny": [
        "Me pretending I understand {t}... until today.",
        "Plot twist: {t} is easier than your group chat drama.",
        "Nobody: ... Me at 2 AM: let's learn about {t}.",
    ],
    "Motivational": [
        "Your future self will thank you for learning about {t} today.",
        "Everyone starts somewhere. Today, let's start with {t}.",
        "Small steps, big change. Let's talk about {t}.",
    ],
    "Awareness": [
        "Stop scrolling for a second. Let's talk about {t}.",
        "Most people ignore this, but {t} matters to all of us.",
```

```
        "This takes less than a minute, and it could change how you see {t}.",
    ],
}

SCENES = {
    "Informative": [
        ("Start with the basics", "First, the basics. When it comes to {t}, begin by understanding the simple idea behind it."),
        ("Why it matters", "Why should you care? Because this shows up in everyday life more than you think."),
        ("Check the source", "Always check where information comes from. Trusted sources beat viral forwards every time."),
        ("Avoid this mistake", "The biggest mistake? Believing the first thing you see. Pause, think, and verify."),
        ("Pro tip", "Pro tip: share what you learn. Teaching others is the fastest way to master it."),
    ],
    "Funny": [
        ("Step one: don't panic", "Step one, don't panic. This is not a boss level. It's the tutorial."),
        ("Step two: ask around", "Step two, ask your friends. At least one of them is secretly an expert."),
        ("Step three: actually try", "Step three, actually try it. Yes, getting off the couch counts as progress."),
        ("Bonus: laugh at mistakes", "Bonus step, laugh at your mistakes. Everyone's first attempt is a little chaotic."),
        ("Final boss: consistency", "Final boss, consistency. Show up every day and it becomes your thing."),
    ],
    "Motivational": [
        ("Start before you're ready", "You don't need to be perfect to begin. You just need to begin."),
        ("Progress over perfection", "Every expert was once a beginner who refused to quit."),
        ("Find your people", "Find a community that cares about {t}. Growth is faster together."),
        ("Make it a habit", "Make it a daily habit. Ten focused minutes beat one perfect day."),
        ("Believe in the process", "Trust the process. One year from now, you'll be glad you started today."),
    ],
    "Awareness": [
        ("Know the facts", "Here are the facts. Understanding {t} is the first step to doing something about it."),
        ("Spot the signs", "Learn to spot the signs early. Awareness helps you act before it is too late."),
        ("Check before you share", "Before you share anything, check it with a reliable source."),
        ("Small actions count", "Small actions count. Talk about it with your friends and family."),
```

```
        ("Be the change", "Be the person who spreads awareness, not confusion."),
    ],
}

CTAS = {
    "Informative": "Follow us on Qoneqt for more quick explainers, and tell us in the comments what you want next.",
    "Funny": "Follow us on Qoneqt for more, and tag that one friend who needs this.",
    "Motivational": "Follow us on Qoneqt and join a community that grows together. Your journey starts now.",
    "Awareness": "Share this on Qoneqt, follow for more, and help us spread awareness.",
}

CAPTIONS = {
    "Informative": "Everything you need to know about {t}, in under a minute.",
    "Funny": "{T}, but make it fun.",
    "Motivational": "Your sign to start with {t} today.",
    "Awareness": "Let's talk about {t}. Awareness starts with you.",
}


def _fmt(s: str, topic: str) -> str:
    cap = topic[:1].upper() + topic[1:]
    return s.replace("{t}", topic).replace("{T}", cap)


STOPWORDS = {"the", "and", "for", "with", "how", "why", "what", "tips", "ways", "about", "your",
             "you", "are", "this", "that", "from", "into", "to", "of", "in", "on", "a", "an"}


def _hashtags(topic: str, tone: str) -> list[str]:
    words = [w for w in re.findall(r"[A-Za-z]+", topic)
             if len(w) > 2 and w.lower() not in STOPWORDS][:3]
    tags = ["#" + "".join(w.capitalize() for w in words)] if words else []
    tags += ["#" + w.capitalize() for w in words if len(w) > 3][:2]
    tags += [f"#{tone}", "#Qoneqt", "#GlobalFeed", "#LearnOnQoneqt", "#ShortVideo"]
    seen, out = set(), []
    for t in tags:
        if t.lower() not in seen and len(t) > 1:
            seen.add(t.lower())
            out.append(t)
    return out[:8]
```

```
def template_script(topic: str, tone: str = "Informative", length: int = 45) -> dict:
    topic = clean_text(topic).rstrip(".!?") or "spotting fake news"
    tone = tone if tone in HOOKS else "Informative"
    n = SCENES_FOR_LENGTH.get(int(length), 4)
    seed = int(hashlib.md5(f"{topic}{tone}".encode()).hexdigest(), 16)
    rng = random.Random(seed)
    lower = topic[:1].lower() + topic[1:] if not topic[:2].isupper() else topic

    scenes = []
    for headline, line in SCENES[tone][:n]:
        scenes.append({
            "narration": _fmt(line, lower),
            "on_screen_text": headline,
            "visual_prompt": f"{topic}, {headline.lower()}, cinematic vertical illustration, vibrant
colors",
        })
    title = topic[:1].upper() + topic[1:]
    if len(title.split()) > 8:
        title = " ".join(title.split()[:8])
    return {
        "title": title,
        "hook": _fmt(rng.choice(HOOKS[tone]), lower),
        "hook_on_screen": title,
        "scenes": scenes,
        "call_to_action": CTAS[tone],
        "caption": _fmt(CAPTIONS[tone], lower) + " Made by TEAM CHAOS' AI agent.",
        "hashtags": _hashtags(topic, tone),
    }


==================== agent/tts.py ====================

"""Text-to-speech voiceover: edge-tts (Microsoft neural voices, free) -> gTTS -> silent track.

edge-tts and gTTS both need internet. If both fail we still produce a silent track of the
right length so the video is always created (the UI shows a warning in that case).
"""
from __future__ import annotations

import asyncio
import os
```

```
from .utils import media_duration, run_ffmpeg

VOICES = {
    "Neerja - Indian English (female)": "en-IN-NeerjaNeural",
    "Prabhat - Indian English (male)": "en-IN-PrabhatNeural",
    "Aria - US English (female)": "en-US-AriaNeural",
    "Andrew - US English (male)": "en-US-AndrewNeural",
    "Sonia - British English (female)": "en-GB-SoniaNeural",
}
DEFAULT_VOICE = "en-IN-NeerjaNeural"


async def _edge_save(text: str, voice: str, path: str, rate: str) -> None:
    import edge_tts

    communicate = edge_tts.Communicate(text, voice, rate=rate)
    await asyncio.wait_for(communicate.save(path), timeout=45)


def _edge_tts(text: str, voice: str, path: str, rate: str = "+5%") -> None:
    try:
        asyncio.run(_edge_save(text, voice, path, rate))
    except RuntimeError as exc:
        # Already inside a running event loop: use a fresh loop in this thread.
        if "event loop" not in str(exc):
            raise
        loop = asyncio.new_event_loop()
        try:
            loop.run_until_complete(_edge_save(text, voice, path, rate))
        finally:
            loop.close()
    if not os.path.exists(path) or os.path.getsize(path) < 1000:
        raise RuntimeError("edge-tts produced no audio")


def _gtts(text: str, path: str) -> None:
    from gtts import gTTS

    gTTS(text=text, lang="en", tld="co.in").save(path)
    if not os.path.exists(path) or os.path.getsize(path) < 1000:
        raise RuntimeError("gTTS produced no audio")
```

```
def _silent(text: str, path: str) -> None:
    seconds = max(2.0, len(text.split()) / 2.5)
    run_ffmpeg(["-f", "lavfi", "-i", "anullsrc=r=44100:cl=stereo", "-t", f"{seconds:.2f}",
                "-c:a", "libmp3lame", "-q:a", "6", path])


def synthesize(text: str, path: str, voice: str = DEFAULT_VOICE,
               engine_preference: str = "edge-tts") -> tuple[str, float, list[str]]:
    """Create an MP3 for `text`. Returns (engine_used, duration_seconds, notes)."""
    notes: list[str] = []
    order = ["edge-tts", "gTTS"] if engine_preference != "gTTS" else ["gTTS", "edge-tts"]
    for engine in order:
        try:
            if engine == "edge-tts":
                _edge_tts(text, voice, path)
            else:
                _gtts(text, path)
            return engine, media_duration(path), notes
        except Exception as exc:
            notes.append(f"{engine} failed: {str(exc)[:200]}")
    _silent(text, path)
    notes.append("All TTS engines failed (no internet?) - used a silent track.")
    return "silent (no TTS available)", media_duration(path), notes


==================== agent/visuals.py ====================

"""Slide rendering with Pillow: 1080x1920 (9:16) frames with burned-in captions.

Optional: AI-generated background images from Pollinations.ai (free, no key needed).
If the image model is slow or offline we fall back to a styled gradient, so it never fails.
"""
from __future__ import annotations

import io
import os
import random
import urllib.parse
from functools import lru_cache

import numpy as np
import requests
from PIL import Image, ImageDraw, ImageEnhance, ImageFilter, ImageFont
```

```
from .utils import FONT_DIR, clean_text

W, H = 1080, 1920
NAVY = (11, 22, 48)
ORANGE = (255, 122, 0)
WHITE = (255, 255, 255)
FOOTER_TEXT = "TEAM CHAOS \u00b7 Qoneqt AI Agent"

# (gradient top, gradient bottom, accent, glow) per tone; several variants per tone.
PALETTES = {
    "Informative": [((11, 22, 48), (18, 70, 110), ORANGE, (0, 170, 255)),
                    ((14, 26, 60), (40, 40, 100), ORANGE, (255, 140, 40))],
    "Funny": [((45, 11, 90), (200, 50, 120), (255, 210, 63), (255, 90, 160)),
              ((30, 14, 70), (255, 95, 60), (255, 230, 90), (120, 80, 255))],
    "Motivational": [((26, 11, 46), (230, 85, 25), (255, 195, 0), (255, 120, 0)),
                     ((12, 20, 45), (150, 40, 20), (255, 175, 40), (255, 80, 40))],
    "Awareness": [((6, 41, 37), (11, 61, 145), (0, 229, 160), (0, 200, 255)),
                  ((10, 30, 50), (20, 90, 90), (0, 229, 160), (60, 140, 255))],
}

POLLINATIONS_URL =
"https://image.pollinations.ai/prompt/{prompt}?width=1080&height=1920&nologo=true&seed={se
ed}"


def palette(tone: str, index: int):
    options = PALETTES.get(tone, PALETTES["Informative"])
    return options[index % len(options)]


@lru_cache(maxsize=64)
def load_font(size: int, bold: bool = True) -> ImageFont.FreeTypeFont:
    names = ["DejaVuSans-Bold.ttf"] if bold else ["DejaVuSans.ttf"]
    candidates = [os.path.join(FONT_DIR, n) for n in names] + [
        os.path.join("/usr/share/fonts/truetype/dejavu", n) for n in names
    ] + ["C:/Windows/Fonts/arialbd.ttf" if bold else "C:/Windows/Fonts/arial.ttf"]
    for path in candidates:
        if os.path.exists(path):
            return ImageFont.truetype(path, size)
    return ImageFont.load_default(size=size)
```

```
def wrap(text: str, font: ImageFont.FreeTypeFont, max_width: int) -> list[str]:
    words, lines, line = text.split(), [], ""
    for word in words:
        test = f"{line} {word}".strip()
        if font.getlength(test) <= max_width or not line:
            line = test
        else:
            lines.append(line)
            line = word
    if line:
        lines.append(line)
    return lines


def fetch_ai_image(prompt: str, seed: int = 42, timeout: int = 45) -> Image.Image | None:
    """Text-to-image via Pollinations.ai (free). Returns None on any failure."""
    try:
        full = f"{clean_text(prompt)[:300]}, vertical 9:16, cinematic lighting, high detail, no text, no
words"
        url = POLLINATIONS_URL.format(prompt=urllib.parse.quote(full), seed=seed)
        resp = requests.get(url, timeout=timeout)
        if resp.status_code != 200 or "image" not in resp.headers.get("content-type", ""):
            return None
        return Image.open(io.BytesIO(resp.content)).convert("RGB")
    except Exception:
        return None


def _cover(img: Image.Image) -> Image.Image:
    scale = max(W / img.width, H / img.height)
    img = img.resize((int(img.width * scale) + 1, int(img.height * scale) + 1), Image.LANCZOS)
    left, top = (img.width - W) // 2, (img.height - H) // 2
    return img.crop((left, top, left + W, top + H))


def make_background(tone: str, index: int, ai_image: Image.Image | None = None) ->
Image.Image:
    top, bottom, accent, glow = palette(tone, index)
    if ai_image is not None:
        bg = ImageEnhance.Brightness(_cover(ai_image)).enhance(0.7).convert("RGBA")
        # Darker gradient overlay so text stays readable on any picture.
        ys = np.linspace(0, 1, H)[:, None]
        alpha = (70 + 120 * np.abs(ys - 0.45) * 2).clip(0, 220)
```

```
        overlay = np.zeros((H, W, 4), dtype=np.uint8)
        overlay[..., :3] = top
        overlay[..., 3] = np.repeat(alpha, W, axis=1).astype(np.uint8)
        return Image.alpha_composite(bg, Image.fromarray(overlay, "RGBA")).convert("RGB")

    ys = np.linspace(0, 1, H)[:, None, None]
    xs = np.linspace(0, 1, W)[None, :, None]
    t = (ys * 0.8 + xs * 0.2) if index % 2 == 0 else (ys * 0.8 + (1 - xs) * 0.2)
    grad = np.array(top) * (1 - t) + np.array(bottom) * t
    bg = Image.fromarray(grad.astype(np.uint8), "RGB").convert("RGBA")

    # Soft glowing blobs (drawn small, blurred, then scaled up - fast).
    rng = random.Random(index * 7 + len(tone))
    small = Image.new("RGBA", (W // 4, H // 4), (0, 0, 0, 0))
    d = ImageDraw.Draw(small)
    for color, alpha in ((glow, 110), (accent, 70)):
        cx, cy = rng.randint(0, W // 4), rng.randint(0, H // 4)
        r = rng.randint(60, 110)
        d.ellipse((cx - r, cy - r, cx + r, cy + r), fill=(*color, alpha))
    small = small.filter(ImageFilter.GaussianBlur(30))
    bg = Image.alpha_composite(bg, small.resize((W, H), Image.BILINEAR))
    return bg.convert("RGB")


def render_base(bg: Image.Image, label: str, headline: str, index: int, total: int,
                tone: str) -> Image.Image:
    """Background + progress bar + label pill + big headline + footer (no caption)."""
    _, _, accent, _ = palette(tone, index)
    img = bg.convert("RGBA")
    layer = Image.new("RGBA", (W, H), (0, 0, 0, 0))
    d = ImageDraw.Draw(layer)

    # Progress bar
    d.rounded_rectangle((60, 60, W - 60, 72), radius=6, fill=(255, 255, 255, 60))
    filled = 60 + int((W - 120) * (index + 1) / max(total, 1))
    d.rounded_rectangle((60, 60, filled, 72), radius=6, fill=(*accent, 255))

    # Label pill, e.g. "SCENE 2 / 4"
    f_label = load_font(42)
    tw = f_label.getlength(label)
    d.rounded_rectangle((60, 120, 60 + tw + 60, 196), radius=38, fill=(*accent, 255))
    d.text((90, 158), label, font=f_label, fill=NAVY, anchor="lm")
```

```
    # Headline, auto-fitted
    headline = clean_text(headline).upper() if len(headline) < 40 else clean_text(headline)
    size, lines = 124, []
    while size >= 60:
        font = load_font(size)
        lines = wrap(headline, font, W - 160)
        line_h = int(size * 1.18)
        if len(lines) <= 5 and line_h * len(lines) <= 760:
            break
        size -= 6
    font = load_font(size)
    line_h = int(size * 1.18)
    block_h = line_h * len(lines)
    y = 820 - block_h // 2
    for line in lines:
        d.text((W // 2 + 4, y + 4), line, font=font, fill=(0, 0, 0, 140), anchor="mt")
        d.text((W // 2, y), line, font=font, fill=WHITE, anchor="mt")
        y += line_h
    d.rounded_rectangle((W // 2 - 90, y + 24, W // 2 + 90, y + 38), radius=7, fill=(*accent, 255))

    # Footer
    f_footer = load_font(34, bold=False)
    d.text((W // 2, H - 90), FOOTER_TEXT, font=f_footer, fill=(255, 255, 255, 210), anchor="mm")
    return Image.alpha_composite(img, layer)


def render_caption_frame(base: Image.Image, caption: str) -> Image.Image:
    """Burn one caption chunk into the lower part of the slide."""
    img = base.copy()
    caption = clean_text(caption)
    if caption:
        layer = Image.new("RGBA", (W, H), (0, 0, 0, 0))
        d = ImageDraw.Draw(layer)
        font = load_font(58)
        lines = wrap(caption, font, W - 220)[:3]
        line_h = 74
        box_h = line_h * len(lines) + 50
        top = 1500 - box_h // 2
        d.rounded_rectangle((70, top, W - 70, top + box_h), radius=30, fill=(0, 0, 0, 175))
        y = top + 25
        for line in lines:
            d.text((W // 2, y), line, font=font, fill=(255, 255, 255, 255), anchor="mt")
            y += line_h
```

```
        img = Image.alpha_composite(img, layer)
    return img.convert("RGB")


def split_captions(text: str, max_words: int = 6) -> list[str]:
    """Split narration into short caption chunks, preferring punctuation breaks."""
    words = clean_text(text).split()
    chunks, cur = [], []
    for w in words:
        cur.append(w)
        if len(cur) >= max_words or (w[-1:] in ".!?,;:" and len(cur) >= 3):
            chunks.append(" ".join(cur))
            cur = []
    if cur:
        if chunks and len(cur) <= 2:
            chunks[-1] += " " + " ".join(cur)
        else:
            chunks.append(" ".join(cur))
    return chunks or [""]


==================== agent/video.py ====================

"""Video assembly with ffmpeg: image frames + voiceover -> H.264/AAC MP4 (plays in
browsers)."""
from __future__ import annotations

import os

from .utils import run_ffmpeg

FPS = 30


def _ffpath(path: str) -> str:
    """Absolute path safe for ffmpeg concat lists (Windows backslashes, quotes)."""
    return os.path.abspath(path).replace("\\", "/").replace("'", "'\\''")


def _concat_list(path: str, entries: list[tuple[str, float]]) -> None:
    with open(path, "w", encoding="utf-8") as fh:
        for img, dur in entries:
            fh.write(f"file '{_ffpath(img)}'\n")
```

```
            fh.write(f"duration {dur:.3f}\n")
        # ffmpeg's concat demuxer needs the last file repeated to honour its duration.
        fh.write(f"file '{_ffpath(entries[-1][0])}'\n")


def build_segment(frames: list[tuple[str, float]], audio_path: str, seconds: float,
                  out_path: str) -> None:
    """One scene: a sequence of caption frames timed to the scene's voiceover."""
    list_path = out_path + ".txt"
    _concat_list(list_path, frames)
    run_ffmpeg([
        "-f", "concat", "-safe", "0", "-i", list_path,
        "-i", audio_path,
        "-filter_complex",
        f"[0:v]fps={FPS},scale=1080:1920,format=yuv420p[v];"
        f"[1:a]aresample=44100,apad[a]",
        "-map", "[v]", "-map", "[a]",
        "-t", f"{seconds:.3f}",
        "-c:v", "libx264", "-preset", "veryfast", "-tune", "stillimage", "-crf", "24",
        "-r", str(FPS), "-g", str(FPS * 2),
        "-c:a", "aac", "-b:a", "128k", "-ac", "2", "-ar", "44100",
        out_path,
    ])


def concat_segments(segment_paths: list[str], out_path: str) -> None:
    """Join scenes into the final video; moov atom up front so browsers stream it fast."""
    list_path = out_path + ".txt"
    with open(list_path, "w", encoding="utf-8") as fh:
        for p in segment_paths:
            fh.write(f"file '{_ffpath(p)}'\n")
    run_ffmpeg([
        "-f", "concat", "-safe", "0", "-i", list_path,
        "-c:v", "copy", "-c:a", "aac", "-b:a", "128k",
        "-movflags", "+faststart",
        out_path,
    ])


==================== agent/utils.py ====================

"""Small helpers: secrets, ffmpeg location, media duration, text cleanup."""
from __future__ import annotations
```

```
import os
import re
import shutil
import subprocess
from functools import lru_cache

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(ROOT_DIR, "outputs")
FONT_DIR = os.path.join(ROOT_DIR, "assets", "fonts")


def get_secret(name: str, default: str = "") -> str:
    """Read a setting from Streamlit secrets first, then environment variables.

    Never hard-code keys: put them in .streamlit/secrets.toml (local) or in the
    Streamlit Community Cloud "Secrets" box, or export them as env variables.
    """
    try:
        import streamlit as st  # imported lazily so the CLI works without a UI

        if name in st.secrets:
            value = str(st.secrets[name]).strip()
            if value:
                return value
    except Exception:
        pass
    return os.environ.get(name, default).strip()


@lru_cache(maxsize=1)
def ffmpeg_exe() -> str:
    """Path to ffmpeg: system install (packages.txt) or the bundled imageio-ffmpeg binary."""
    path = shutil.which("ffmpeg")
    if path:
        return path
    try:
        import imageio_ffmpeg

        return imageio_ffmpeg.get_ffmpeg_exe()
    except Exception as exc:  # pragma: no cover
        raise RuntimeError(
            "ffmpeg was not found. Install it (see README) or `pip install imageio-ffmpeg`."
```

```
        ) from exc


def ffprobe_exe() -> str | None:
    return shutil.which("ffprobe")


def media_duration(path: str) -> float:
    """Duration of an audio/video file in seconds."""
    probe = ffprobe_exe()
    if probe:
        try:
            out = subprocess.run(
                [probe, "-v", "error", "-show_entries", "format=duration",
                 "-of", "default=noprint_wrappers=1:nokey=1", path],
                capture_output=True, text=True, timeout=30,
            )
            return float(out.stdout.strip())
        except Exception:
            pass
    # Fallback: parse "Duration: 00:00:05.12" from ffmpeg's banner.
    out = subprocess.run([ffmpeg_exe(), "-i", path], capture_output=True, text=True, timeout=30)
    match = re.search(r"Duration:\s*(\d+):(\d+):(\d+(?:\.\d+)?)", out.stderr)
    if not match:
        raise RuntimeError(f"Could not read duration of {path}")
    h, m, s = match.groups()
    return int(h) * 3600 + int(m) * 60 + float(s)


def run_ffmpeg(args: list[str], timeout: int = 300) -> None:
    """Run ffmpeg and raise a readable error if it fails."""
    cmd = [ffmpeg_exe(), "-hide_banner", "-loglevel", "error", "-y", *args]
    proc = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
    if proc.returncode != 0:
        raise RuntimeError("ffmpeg failed: " + (proc.stderr.strip()[-800:] or "unknown error"))


_EMOJI_RE = re.compile(

"[\U0001F000-\U0001FAFF\U00002600-\U000027BF\U0001F900-\U0001F9FF\U0000FE0F\U0
000200D]+"
)
```

```
def clean_text(text: str) -> str:
    """Remove emojis / odd characters that fonts and TTS can't handle; collapse spaces."""
    text = _EMOJI_RE.sub("", str(text or ""))
    text = text.replace("\u2019", "'").replace("\u2018", "'")
    text = text.replace("\u201c", '"').replace("\u201d", '"')
    text = text.replace("\u2013", "-").replace("\u2014", " - ")
    return re.sub(r"\s+", " ", text).strip()


def slugify(text: str, max_len: int = 40) -> str:
    slug = re.sub(r"[^a-zA-Z0-9]+", "-", text.lower()).strip("-")
    return (slug[:max_len].strip("-")) or "video"


==================== requirements.txt ====================

# Qoneqt AI Video Agent - Python 3.11 / 3.12 / 3.13
streamlit==1.65.0
edge-tts==7.2.8
gTTS==2.5.4
Pillow==12.3.0
requests==2.34.2
numpy>=1.26
# Backup ffmpeg binary if the system ffmpeg (packages.txt) is missing
imageio-ffmpeg==0.6.0


==================== packages.txt ====================

ffmpeg
fonts-dejavu-core


==================== .streamlit/config.toml ====================

[theme]
base = "dark"
primaryColor = "#FF7A00"
backgroundColor = "#0B1630"
secondaryBackgroundColor = "#13244A"
textColor = "#F5F7FB"
```

```
[browser]
gatherUsageStats = false

[server]
maxUploadSize = 50
```

========== .streamlit/secrets.toml.example ==========

```
# Copy this file to .streamlit/secrets.toml (local) - it is git-ignored, never commit it.
# On Streamlit Community Cloud paste the same lines into: App -> Settings -> Secrets.

# Free key from https://console.groq.com/keys
GROQ_API_KEY = "gsk_your_key_here"
# GROQ_MODEL = "llama-3.3-70b-versatile"

# Optional alternative LLM - free key from https://aistudio.google.com/apikey
# GEMINI_API_KEY = "your_gemini_key_here"
# GEMINI_MODEL = "gemini-2.5-flash"
```

========== .gitignore ==========

```
# Secrets - never commit keys
.streamlit/secrets.toml
.env
*.env

# Python
venv/
.venv/
env/
__pycache__/
*.pyc

# Generated videos
outputs/
*.mp4
!sample_output/sample.mp4

# OS / editors
.DS_Store
Thumbs.db
```

.vscode/
.idea/